

UNIVERSIDAD DEL VALLE DE GUATEMALA

Redes

Kevin Antonio Velasquez Aguilar

Sección: 11



Laboratorio 1: Esquemas de comunicación

Mario Fernando Rocha – 23501

Guatemala 14 de Julio del 2026

Segunda parte: Introducción a Wireshark

Reporte individual — Laboratorio 1: Esquemas de comunicación e introducción a Wireshark

En esta parte trabajé de forma individual con Wireshark: personalicé el entorno, configuré una captura con ring buffer y analicé tráfico HTTP real generado por mí. A continuación documento cada paso con la evidencia correspondiente y las decisiones técnicas detrás de cada configuración.

3.4 Personalización del entorno

El objetivo de esta sección era dejar el entorno de Wireshark configurado a mi gusto y separado de la configuración por defecto, usando el archivo intro-wireshark-trace1.pcap proporcionado en Canvas como base para practicar sin necesidad de generar tráfico propio todavía.

Perfil de configuración

Lo primero fue crear un perfil personalizado desde **Edit → Configuration Profiles**, con mi nombre y apellido. Los perfiles en Wireshark guardan de forma independiente las columnas, colores, layout y filtros que uno configura, así que sirven para tener un “ambiente” propio sin pisar la configuración por defecto ni la de otros perfiles (Bluetooth, Classic, etc.) que ya vienen instalados.

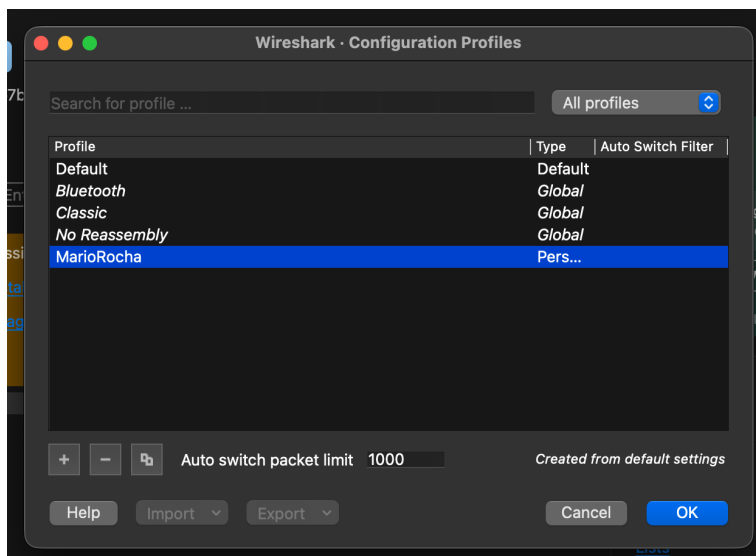


Figura 1. Perfil “MarioRocha” creado en Configuration Profiles.

Apertura del archivo de práctica

Descargué intro-wireshark-trace1.pcap desde Canvas y lo abrí directamente desde **File → Open**. Este archivo trae 651 registros de tráfico ya capturado, lo que permite practicar el resto de la personalización sin depender de generar tráfico en vivo.

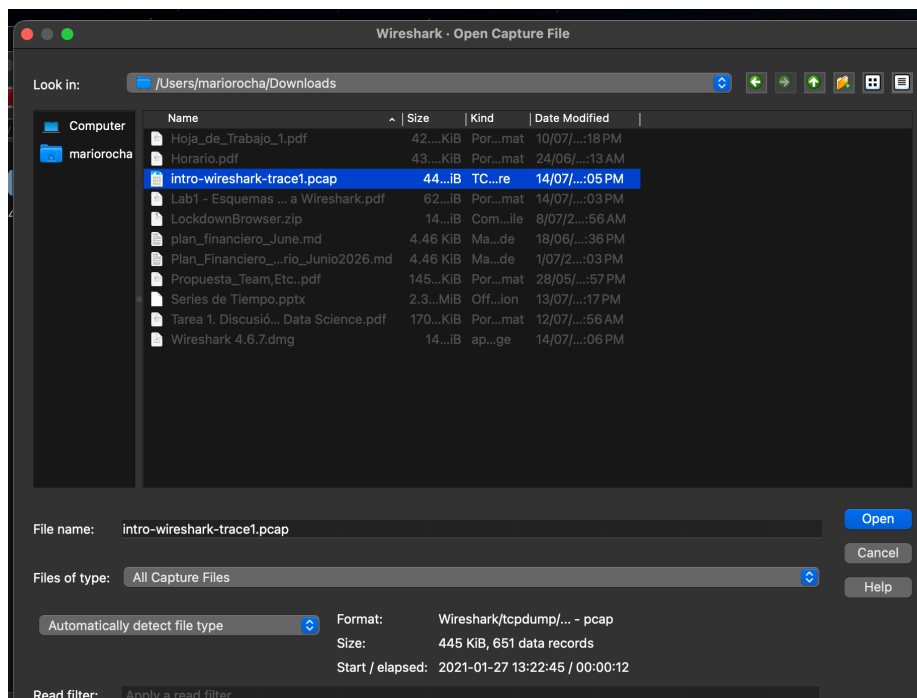


Figura 2. Apertura de intro-wireshark-trace1.pcap desde el diálogo Open Capture File.

Formato de hora

Cambié el formato de tiempo a **Time of Day** desde **View** → **Time Display Format**. Por defecto Wireshark muestra el tiempo como segundos transcurridos desde el primer paquete capturado, lo cual es útil para medir duraciones relativas, pero no dice nada sobre la hora real en que ocurrió cada evento. Con Time of Day cada paquete queda con su hora real (hh:mm:ss), que es mucho más natural para correlacionar la captura con lo que uno estaba haciendo en ese momento.

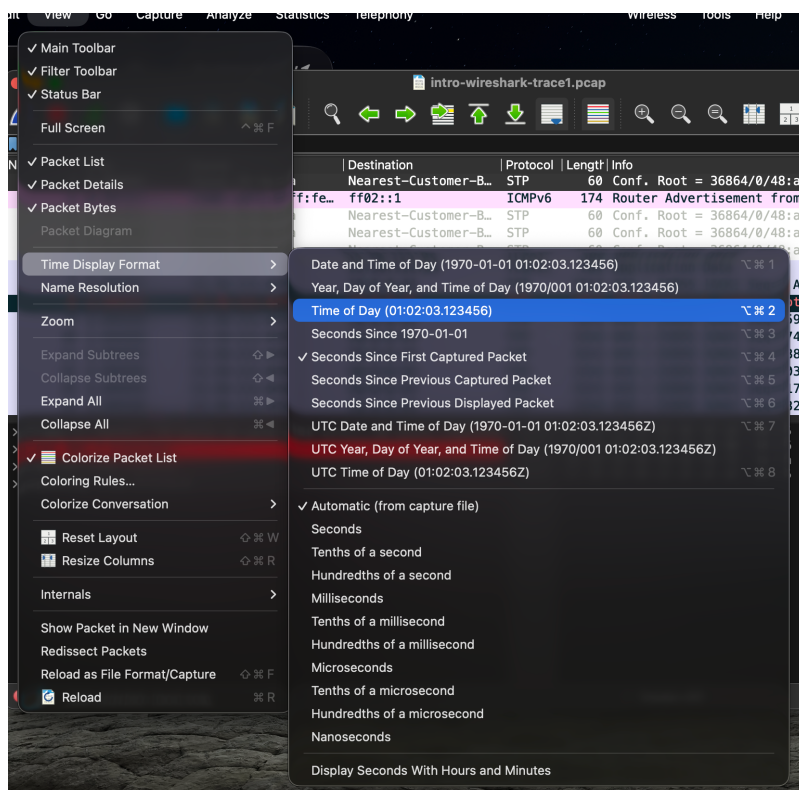


Figura 3. Selección de Time of Day en el menú View → Time Display Format.

Columna de longitud de protocolo

Agregué una columna nueva desde **Edit** → **Preferences** → **Appearance** → **Columns** para mostrar la longitud del protocolo, y después eliminé la columna Length que trae Wireshark por defecto. La diferencia entre ambas no es cosmética: Length es el tamaño total del frame capturado a nivel de Ethernet, mientras que la longitud de protocolo refleja el tamaño del segmento de la capa que realmente interesa analizar (TCP, HTTP, etc.). Para un análisis enfocado en el protocolo de aplicación, la segunda es más útil que el tamaño físico del paquete completo.

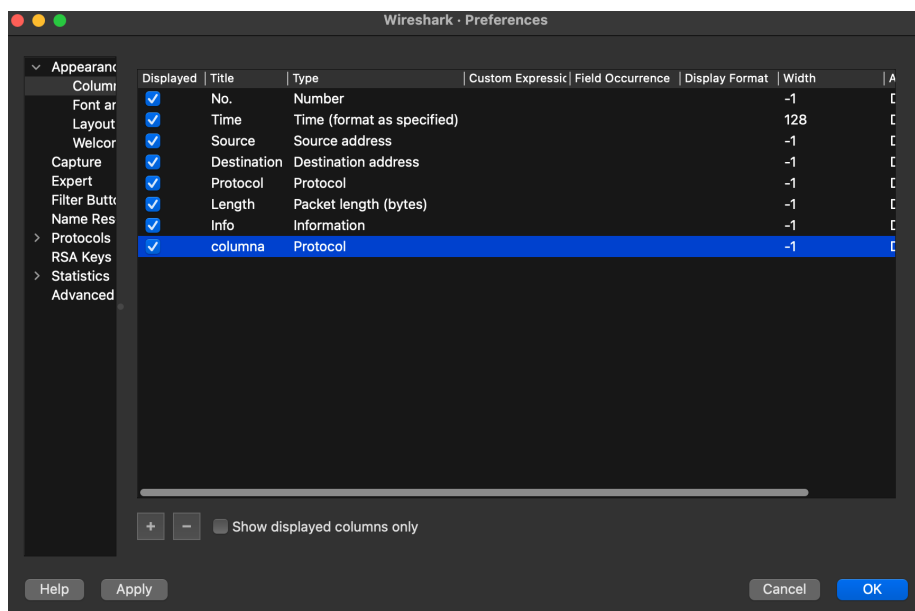


Figura 4. Columna “columna” (Protocol) agregada en Preferences → Columns.

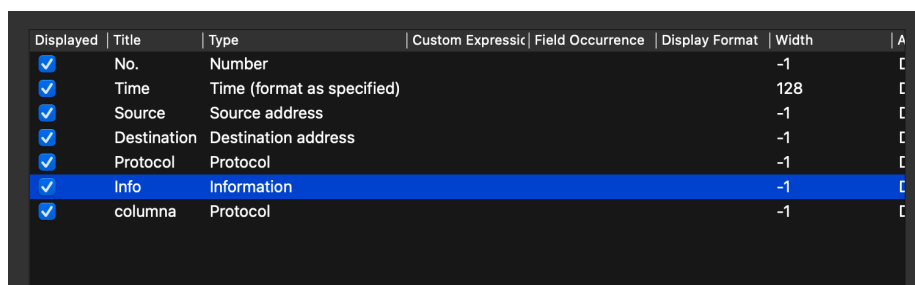


Figura 5. Listado de columnas final, sin Length y con la columna de protocolo agregada.

Layout de paneles

Cambié la distribución de los tres paneles (lista de paquetes, detalle y bytes) desde **Preferences** → **Layout**, eligiendo una disposición distinta a la vertical apilada que trae por defecto. El layout no afecta los datos, solo cómo se distribuyen en pantalla, pero elegir uno que aproveche mejor el ancho de mi monitor hace más cómodo revisar el detalle de un paquete sin perder de vista la lista completa.

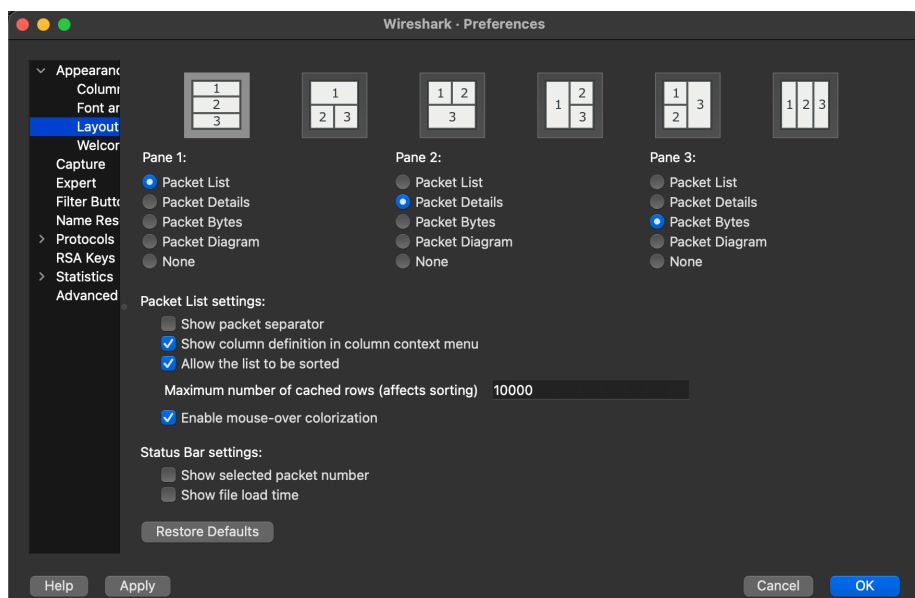


Figura 6. Selección de distribución de paneles en Preferences → Layout.

Regla de color para TCP con SYN = 1

Desde **View** → **Coloring Rules** creé una regla nueva llamada TCP_new con el filtro `tcp.flags.syn == 1`, que resalta cualquier paquete TCP con la bandera SYN activada. Vale la pena aclarar que ese filtro marca tanto el primer paquete de un handshake (SYN puro) como la respuesta del servidor (SYN-ACK), porque ambos tienen esa bandera en 1; si se quisiera aislar únicamente el SYN inicial habría que combinarlo con `tcp.flags.ack == 0`.

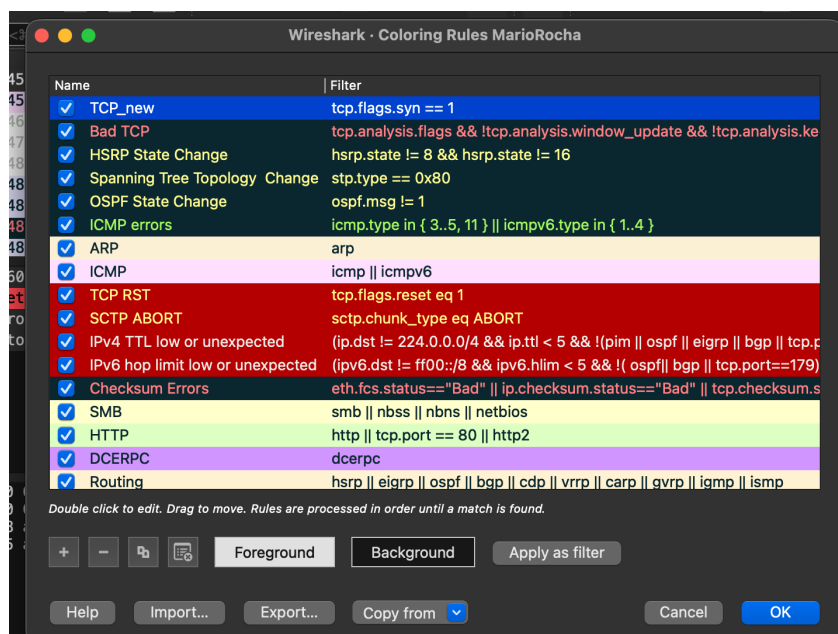


Figura 7. Regla de color TCP_new (`tcp.flags.syn == 1`) en Coloring Rules.

Botón de filtro rápido

Con el signo + junto a la barra de filtros creé un botón llamado TCP SYN que aplica automáticamente el mismo filtro `tcp.flags.syn == 1`. Al probarlo sobre una captura, deja visibles únicamente los paquetes del handshake correspondientes, como se ve en el ejemplo entre 10.0.0.44 y 128.119.245.12.

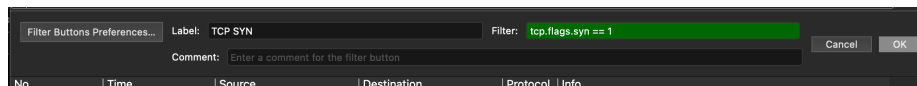


Figura 8a. Creación del botón de filtro TCP SYN.

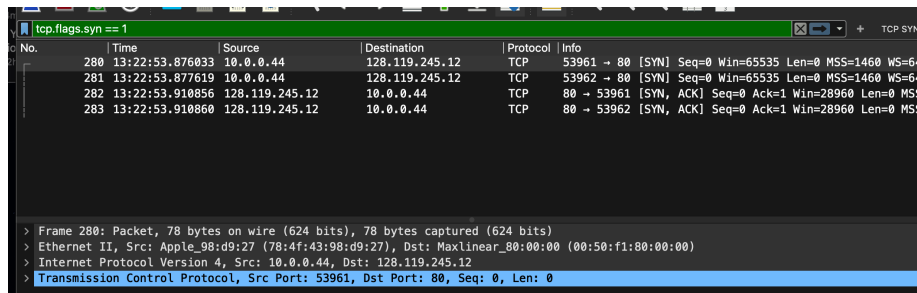


Figura 8b. Resultado del filtro aplicado mediante el botón.

Interfaces virtuales ocultas

En **Capture** → **Options** → **Manage Interfaces** desmarqué todas las interfaces que no corresponden a mi conexión física real: `utun0`–`utun5`, `awdl0`, `llw0`, Loopback, los puertos Thunderbolt sin usar, `anpi0/1`, `gif0` y `stf0`. macOS expone por defecto un montón de interfaces virtuales (túneles VPN, AirDrop/Handoff, loopback, interfaces internas de Apple Silicon) que casi nunca son relevantes para un análisis normal de red, así que dejarlas ocultas simplifica bastante la lista al momento de elegir dónde capturar.

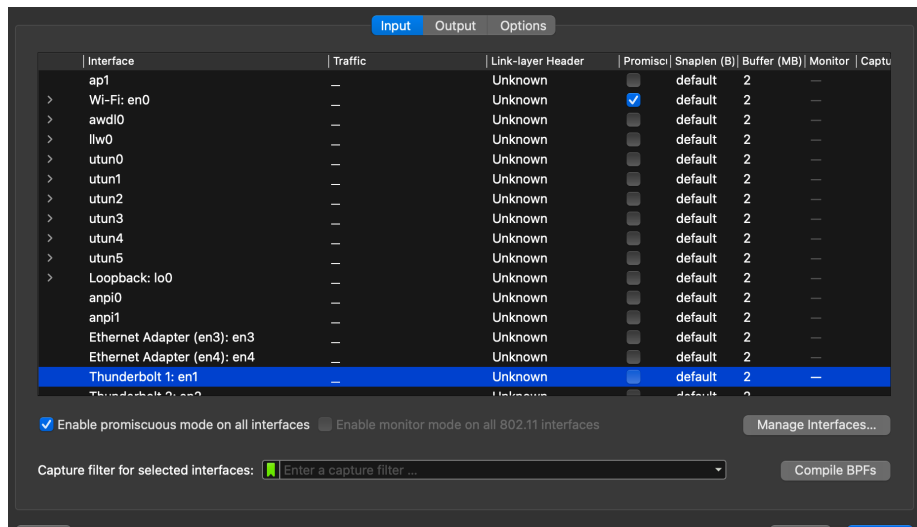


Figura 9. Interfaces virtuales desmarcadas en Manage Interfaces.

El resultado final deja visible únicamente `Wi-Fi: en0`, que es mi interfaz inalámbrica real:

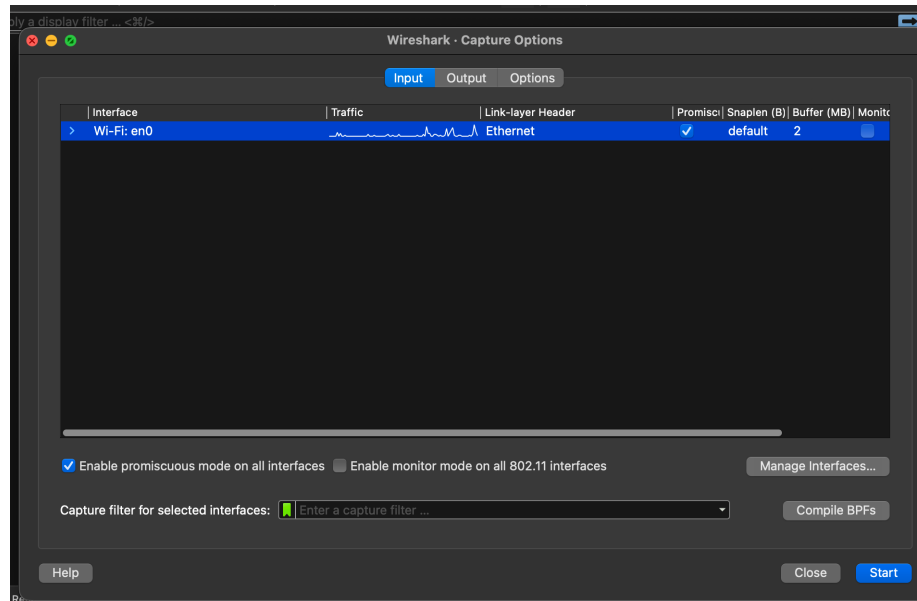


Figura 10. Lista de interfaces de captura simplificada, mostrando únicamente Wi-Fi: en0.

3.5 Configuración de la captura de paquetes

Revisión de la interfaz con ifconfig

Antes de configurar la captura corrí `ifconfig en0` en terminal para revisar el estado de mi interfaz de red:

```
$ ifconfig en0
en0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
options=6460<TSO4,TSO6,CHANNEL_IO,PARTIAL_CSUM,ZEROINVERT_CSUM>
ether 9a:50:d3:7f:69:1e
inet6 fe80::18d5:6ba8:719f:ed4%en0 prefixlen 64 secured scopeid 0xb
inet6 2803:c800:40c9:fd87:102f:c34c:1847:e442 prefixlen 64 autoconf secured
inet6 2803:c800:40c9:fd87:a114:6351:f38d:cda1 prefixlen 64 autoconf temporary
inet 10.204.56.195 netmask 0xffffffff broadcast 10.204.56.255
nd6 options=201<PERFORMNUD,DAD>
media: autoselect
status: active
```

La bandera UP y el status: active confirman que la interfaz está físicamente conectada y funcionando. La línea ether muestra mi dirección MAC (identificador de capa 2), mientras que inet muestra la IPv4 asignada localmente (10.204.56.195) bajo una máscara /24 (255.255.255.0). Las tres líneas inet6 corresponden a IPv6: la primera es una dirección *link-local* (solo válida dentro de mi red local, nunca sale a internet), y las otras dos son direcciones globales generadas automáticamente (autoconf); una de ellas está marcada como temporary, que es una función de privacidad de macOS que rota periódicamente la IPv6 pública para que el dispositivo no sea rastreable a largo plazo usando siempre la misma dirección.

Configuración del ring buffer

En **Capture** → **Options** → **Output** definí el archivo de salida lab1_23501.pcap, activé Create a new file automatically con un límite de **5 megabytes** por archivo, y activé Use a ring buffer con un máximo de **10 archivos**. La idea del ring buffer es que, en vez de un solo archivo que crece

indefinidamente, Wireshark va rotando entre varios archivos de tamaño fijo: al llenarse el archivo número 10 empieza a sobrescribir el número 1, y así sucesivamente. Esto es útil para capturas largas donde no interesa conservar todo el historial completo, sino solo la actividad más reciente, sin arriesgarse a llenar el disco.

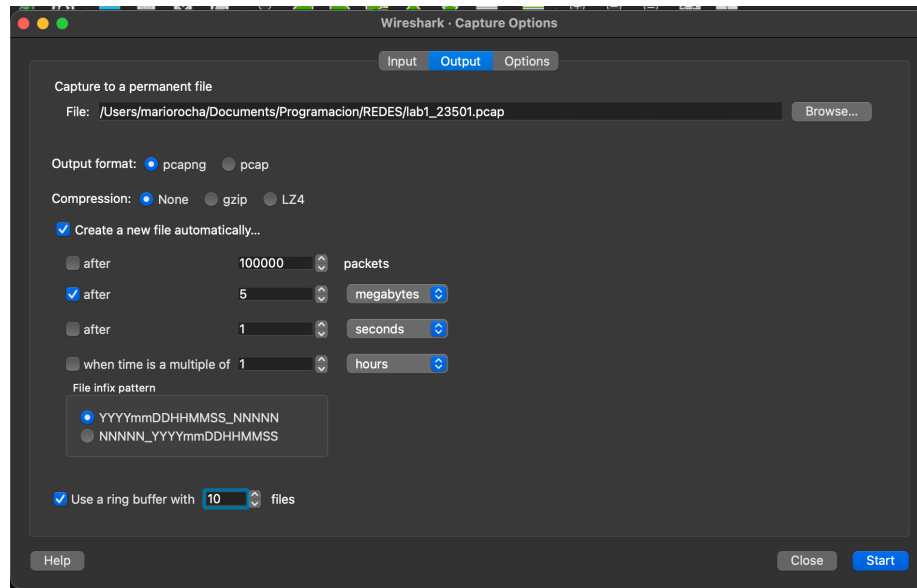


Figura 11. Configuración de salida con ring buffer de 5 MB y 10 archivos.

Después de generar tráfico navegando un rato, confirmé el comportamiento revisando la carpeta de destino: los archivos se numeran automáticamente (lab1_23501_<timestamp>_00001.pcap, _00002.pcap, etc.) y, al superar el límite configurado, Wireshark efectivamente empezó a borrar los archivos más antiguos para mantener siempre un máximo de 10, confirmando que el mecanismo circular funciona como se espera.

Name	Date Modified	Size	Kind
lab1_23501_20260...94144_00007.pcap	Today at 7:42 PM	5 MB	Pcap N...apture
lab1_23501_20260...94204_00008.pcap	Today at 7:42 PM	5 MB	Pcap N...apture
lab1_23501_20260...94223_00009.pcap	Today at 7:42 PM	5 MB	Pcap N...apture
lab1_23501_20260...94240_00010.pcap	Today at 7:42 PM	5 MB	Pcap N...apture
lab1_23501_20260...94252_00011.pcap	Today at 7:43 PM	5 MB	Pcap N...apture
lab1_23501_202...4308_00012.pcap	Today at 7:43 PM	5 MB	Pcap N...apture
lab1_23501_202...4322_00013.pcap	Today at 7:43 PM	↑ 1.2 MB	Pcap N...apture
lab1_23501_202...4335_00014.pcap	Today at 7:43 PM	5 MB	Pcap N...apture
lab1_23501_202...4347_00015.pcap	Today at 7:44 PM	5 MB	Pcap N...apture
lab1_23501_202...4403_00016.pcap	Today at 7:44 PM	↑ 547 bytes	Pcap N...apture

Figura 12. Archivos .pcap generados por el ring buffer en Finder.

3.6 Análisis de paquetes

Para esta parte tuve que ajustar el método de captura sobre la marcha. Intenté varias veces cargar la página del laboratorio desde el navegador (Safari, en ventana privada, escribiendo explícitamente http:// en la URL) y, aun así, Wireshark seguía mostrando el tráfico como TLS en vez de HTTP en texto plano. Después de descartar temas de caché y de resolución DNS, identifiqué la causa: varios navegadores modernos fuerzan automáticamente el upgrade de conexiones HTTP a HTTPS por seguridad, incluso

cuando el sitio de destino no lo soporta bien, lo cual hace que a nivel de red el tráfico salga cifrado sin que el usuario lo note en la barra de direcciones.

Para eliminar esa capa de complejidad y capturar el protocolo HTTP de forma directa, usé curl desde terminal como cliente HTTP, con la captura de Wireshark corriendo en simultáneo:

```
$ curl http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html
<html>
Congratulations! You've downloaded the first Wireshark lab file!
</html>
```

Con eso sí se capturó el intercambio HTTP completo: el GET del cliente y la respuesta 200 OK del servidor, ambos en texto plano y visibles al aplicar el filtro http.

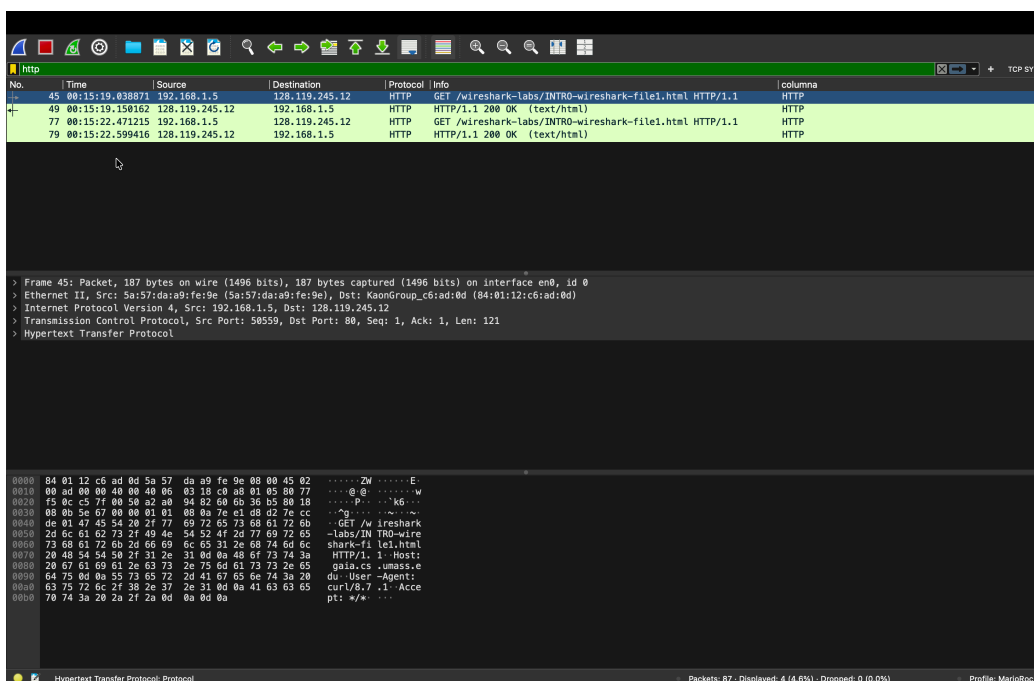


Figura 13a. Captura filtrada por http mostrando el GET (paquete 45) y la respuesta 200 OK (paquete 49).

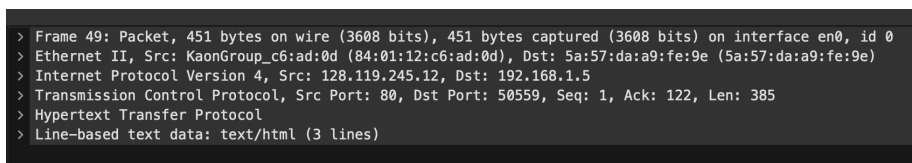


Figura 13b. Detalle expandido del paquete 49 (respuesta HTTP/1.1 200 OK).

Preguntas de análisis

a) ¿Qué versión de HTTP está ejecutando el navegador (cliente)?

HTTP/1.1, visible en la línea de petición del paquete 45 (“GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1”).

b) ¿Qué versión de HTTP está ejecutando el servidor?

HTTP/1.1, visible en la respuesta del paquete 49 (“HTTP/1.1 200 OK”).

c) ¿Qué lenguajes indica el navegador que acepta el servidor?

No se pudo determinar en esta captura: al usar curl en vez de un navegador real, la petición no incluyó automáticamente el encabezado Accept-Language, ya que curl solo lo envía si se especifica explícitamente con -H. Un navegador normal sí lo incluye por defecto con los idiomas configurados en el sistema operativo.

d) ¿Cuántos bytes de contenido fueron devueltos por el servidor?

81 bytes, según el encabezado Content-Length: 81 en la respuesta del paquete 49.

e) Ante un problema de rendimiento, ¿en qué elementos de la red convendría escuchar los paquetes? ¿Es conveniente instalar Wireshark en el servidor?

Lo más útil sería capturar tanto en el cliente como en los routers o switches intermedios de la ruta, ya que ahí se pueden identificar retransmisiones, pérdida de paquetes o cuellos de botella específicos. Instalar Wireshark directamente en el servidor, en cambio, no es lo más conveniente: normalmente no se tiene acceso administrativo a servidores de terceros, capturar ahí expondría el tráfico de todos los usuarios conectados (un problema serio de privacidad), y mantener un proceso de captura constante en un servidor de producción añade overhead justo en el componente que se está intentando diagnosticar.

Dato adicional: los mismos encabezados de la respuesta revelan que el servidor corre Apache/2.4.62 sobre AlmaLinux con OpenSSL/3.5.5, mod_fcgi y mod_perl/2.0.12, y que el archivo no se modifica desde octubre de 2025 (Last-Modified), consistente con tratarse de contenido estático de un laboratorio universitario que rara vez cambia.

Discusión

Primera parte — Esquemas de comunicación

En esta parte, trabajando junto con Genser, primero probamos los esquemas clásicos de codificación: Morse y Baudot. Morse terminó siendo mucho más fácil de transmitir y decodificar en tiempo real: al usar pulsos de duración variable (puntos y rayas), el oído agarra el ritmo de forma bastante natural, y las pausas entre letras y palabras ayudan a no perderse. Baudot, al ser un código de longitud fija (5 bits por carácter), exige una sincronización perfecta; bastaba con adelantarse o atrasarse un solo bit para desfasar la letra completa y arruinar el resto del mensaje, lo que se tradujo directamente en más errores y más repeticiones. Ese contraste deja bastante claro un patrón que se repite en redes reales: los esquemas rígidos de longitud fija son ideales para que los procese una máquina, donde el reloj es exacto, pero son mucho más frágiles cuando quien transmite y recibe es una persona.

La segunda parte, con los mensajes “empaquetados” en notas de voz en vez de tiempo real, mostró otra cara del mismo problema: sin poder pedir “repíte” a mitad de la transmisión, cualquier error obligaba a reescuchar el audio completo desde el inicio. Es básicamente la diferencia entre un canal con retroalimentación inmediata y uno sin ella, algo que en redes reales se traduce en la diferencia entre protocolos con confirmación en tiempo real y protocolos que dependen de retransmitir el mensaje completo cuando algo se pierde.

Con la parte de conmutación (tres clientes y un conmutador), lo más interesante fue diseñar un protocolo mínimo para coordinar quién le habla a quién sin que los mensajes chocaran entre sí: usamos una palabra

clave para indicar el destino y un sistema simple de turnos (“DISPONIBLE” / “Recibido”) para que el conmutador no recibiera audios simultáneos de varios clientes. Aunque suena básico, es justo el tipo de problema que resuelven en la práctica los protocolos de control de acceso al medio y las tablas de reenvío de un switch real.

Segunda parte — Wireshark

La personalización del entorno (perfil, columnas, layout, reglas de color, botón de filtro) fue bastante directa una vez que entendí en qué submenú de Preferences vivía cada opción.

Donde sí me trabé fue al iniciar la primera captura: macOS bloqueó el acceso a la interfaz con un error de permisos sobre el dispositivo BPF (Berkeley Packet Filter), el mecanismo del kernel que permite leer tráfico crudo de la red. La solución fue instalar el paquete ChmodBPF que viene incluido con Wireshark, el cual crea un grupo de sistema (`access_bpf`) y agrega al usuario actual a ese grupo, y después reiniciar la Mac para que el cambio de permisos tomara efecto. Es un paso que la guía del laboratorio no menciona, pero que resultó prácticamente obligatorio en una instalación reciente de macOS para poder capturar sin recurrir a `sudo`.

La configuración del ring buffer no dio mayor problema, y confirmar que funcionaba fue sencillo: después de generar tráfico un rato, vi cómo Wireshark iba rotando los archivos y borrando los más viejos al superar el límite de 10 configurado.

Donde más aprendí, sin buscarlo, fue en la parte de capturar tráfico HTTP puro. Como describo en la sección 3.6, el navegador estaba subiendo automáticamente la conexión a HTTPS a pesar de que yo escribía explícitamente `http://` en la URL, un comportamiento de seguridad que varios navegadores activan por defecto y que no siempre es obvio para quien lo usa. Cambiar a curl como cliente eliminó cualquier ambigüedad y me permitió capturar exactamente el GET y el 200 OK en texto plano que necesitaba. Fue un buen recordatorio de que, aunque uno escriba “http” en la barra de direcciones, el navegador puede estar tomando decisiones de seguridad por debajo que cambian por completo lo que efectivamente viaja por la red — y de que Wireshark es precisamente la herramienta que deja esa clase de comportamiento en evidencia.

Conclusiones

- Los esquemas de codificación de longitud variable, como Morse, son más tolerantes a errores humanos que los de longitud fija, como Baudot, porque no dependen de una sincronización perfecta bit a bit.
- La comunicación “empaquetada” sin retroalimentación inmediata multiplica el costo de cualquier error, ya que obliga a repetir la transmisión completa en vez de corregir sobre la marcha.
- Introducir un conmutador simplifica la topología de la red, pasando de conexiones punto a punto a un solo intermediario, pero exige definir un protocolo explícito de direccionamiento y control de acceso al medio para evitar colisiones.
- El comportamiento “de seguridad por defecto” de un navegador (upgrade automático a HTTPS) puede cambiar por completo lo que se observa en una captura de red, incluso cuando la URL escrita

dice explícitamente `http://`; Wireshark es la herramienta que permite detectar ese tipo de discrepancias entre lo que el usuario cree que está pasando y lo que realmente viaja por la red.

- Los permisos a nivel de sistema operativo, como el acceso a `/dev/bpf` en macOS, son parte real del trabajo de análisis de red y no un simple detalle técnico: sin resolverlos, ninguna herramienta de captura funciona sin importar qué tan bien esté configurada.

Referencias

Wireshark Foundation. (s.f.). *Installing Wireshark under macOS*. Wireshark User's Guide.

https://www.wireshark.org/docs/wsug_html_chunked/ChBuildInstallOSXInstall.html

Wireshark Foundation. (s.f.). *CaptureSetup/CapturePrivileges*. Wireshark Wiki.

<https://wiki.wireshark.org/CaptureSetup/CapturePrivileges>

Fielding, R., & Nottingham, M. (Eds.). (2022). *RFC 9112: HTTP/1.1*. IETF. <https://www.rfc-editor.org/rfc/rfc9112>

curl. (s.f.). *curl(1) command line tool — Manual*. <https://curl.se/docs/manpage.html>

Apple Inc. (s.f.). *ifconfig(8) — macOS/BSD manual pages*.